

DSA Trainer Job Tracker

Code Walkthrough -- every authored line, explained

Repository: github.com/yogesh889/DSA_Trainer_Online

Live page: yogesh889.github.io/DSA_Trainer_Online

This document walks through every file this project's automation actually runs -- `scrape.py`, `generate_report.py`, both GitHub Actions workflows, and `requirements.txt` -- one line (or one short block of blank/structural lines) at a time. Generated data files (`jobs.json`, `jobs.md`, `index.html`, `report_state.json`, `latest_report.pdf`) are produced by that code rather than hand-written, and are listed at the end instead of walked through line-by-line.

scrape.py

The scraper. Runs every ~5 hours via GitHub Actions: drives headless Chrome against LinkedIn's public, logged-out job-search endpoint, parses results, deduplicates them, applies the 1-to-31-day freshness window, and writes jobs.json / jobs.md / index.html.

```
L1:  
#!/usr/bin/env python3  
-> Shebang, so the file can also be run directly as ./scrape.py on a Unix-like system.
```

```
L2-9:  
"""  
Tracks "DSA Trainer" job postings in India from LinkedIn's public,  
logged-out job search. No login, no credentials, no authenticated  
session is ever used -- only the guest job-search endpoint that LinkedIn  
serves to anonymous visitors.  
  
Run on a schedule (see .github/workflows/dsa-trainer-job-watch.yml).  
"""
```

-> Module docstring: states the project's core constraint up front -- only LinkedIn's public, logged-out guest search is ever touched, no login or credentials, and points to the workflow file that schedules this script.

```
L10-16:  
import json  
import re  
import sys  
import time  
import urllib.parse  
from pathlib import Path  
from datetime import datetime, timezone
```

-> Standard-library imports: json (read/write jobs.json), re (parsing job ids and relative "posted" text), sys (stderr for warnings), time (the inter-page delay), urllib.parse (building the query string), Path (filesystem paths), datetime/timezone (UTC timestamps).

```
L17: (blank)
```

-> Blank line separating standard-library imports from third-party imports.

```
L18-20:  
from bs4 import BeautifulSoup  
from selenium import webdriver  
from selenium.webdriver.chrome.options import Options as ChromeOptions
```

-> Third-party imports: BeautifulSoup parses the HTML LinkedIn returns; webdriver and ChromeOptions (from Selenium) drive a real headless Chrome browser instead of a raw HTTP request.

L21: (blank)

-> Blank line before the path constants.

L22:

```
REPO_DIR = Path(__file__).resolve().parent
```

-> REPO_DIR resolves to the directory this script itself lives in, so output paths are correct no matter what directory the script is invoked from (important under GitHub Actions).

L23-25:

```
JOBS_JSON = REPO_DIR / "jobs.json"
```

```
JOBS_MD = REPO_DIR / "jobs.md"
```

```
INDEX_HTML = REPO_DIR / "index.html"
```

-> The three output files this script maintains: the machine-readable JSON record, the human-readable markdown table, and the static HTML page GitHub Pages serves.

L26: (blank)

-> Blank line before the search parameters.

L27-28:

```
KEYWORDS = "DSA Trainer"
```

```
LOCATION = "India"
```

-> The exact search query sent to LinkedIn: keywords "DSA Trainer", location "India" -- the single place to change if the tracked search should change.

L29:

```
GUEST_SEARCH_URL = "https://www.linkedin.com/jobs-guest/jobs/api/seeMoreJobPostings/search"
```

-> GUEST_SEARCH_URL: LinkedIn's public jobs-guest search API -- the same endpoint LinkedIn serves to anonymous visitors and search-engine crawlers, requiring no session at all.

L30-32:

```
PAGE_SIZE = 10
```

```
MAX_PAGES = 6 # LinkedIn's guest endpoint stops returning new results after ~40-60 for a given query
```

```
REQUEST_DELAY_SECONDS = 2
```

-> Pagination and pacing constants: 10 results per page, up to 6 pages, with a 2-second pause between page loads so the scraper behaves politely rather than hammering the endpoint.

L33: (blank)

-> Blank line before the user-agent string.

L34-37:

```
USER_AGENT = (
```

```
    "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 "
```

```
    "(KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
```

```
)
```

-> USER_AGENT: an ordinary desktop Chrome string, so requests look like a normal browser visit rather than a bare script -- passed to Chrome itself via build_driver() below.

L38: (blank)

-> Blank line before the freshness-window constants.

L39-40:

```
MIN_AGE_DAYS = 1
MAX_AGE_DAYS = 31 # ~1 month
```

-> *MIN_AGE_DAYS / MAX_AGE_DAYS: a listing is only tracked once it is at least 1 day old, and is dropped again once it is older than about a month (31 days).*

L41: (blank)

-> *Blank line before the unit-conversion table.*

```
L42-49:
_UNIT_TO_DAYS = {
    "minute": 1 / 1440,
    "hour": 1 / 24,
    "day": 1,
    "week": 7,
    "month": 30,
    "year": 365,
}
```

-> *_UNIT_TO_DAYS: converts each unit LinkedIn's relative "posted" text can use (minute, hour, day, week, month, year) into a fraction of a day, so any of them can be turned into a single comparable number.*

L50-51: (blank)

-> *Two blank lines -- PEP 8 spacing before the first function definition.*

```
L52-53:
def parse_age_days(posted_text: str):
    """Approximate age in days from LinkedIn's relative 'posted' text, or None if unparseable."""
```

-> *parse_age_days: function signature and docstring -- turns LinkedIn's relative text into an approximate age in days, or None if it can't be parsed.*

```
L54:
    text = (posted_text or "").strip().lower()
```

-> *Normalizes the input: falls back to an empty string if None was passed, strips whitespace, lowercases it so matching below is case-insensitive.*

```
L55-56:
    if not text:
        return None
```

-> *An empty string can't be parsed -- return None immediately.*

```
L57-58:
    if "just now" in text or "moment" in text:
        return 0.0
```

-> *LinkedIn's "just now" / "a moment ago" phrasing is treated as age zero.*

```
L59-60:
    if "yesterday" in text:
        return 1.0
```

-> "Yesterday" is treated as exactly 1 day old (LinkedIn doesn't say "1 day ago" for this case).

```
L61:
    match = re.search(r"(\d+)\s*(minute|hour|day|week|month|year)s?\s*ago", text)
```

-> The general case: a regex pulls out a number and a unit word from phrasing like "3 days ago" or "2 weeks ago" (the trailing "s?" makes the plural optional, so "1 day ago" also matches).

```
L62-63:
    if not match:
        return None
```

-> If the text didn't match that pattern at all (an LinkedIn phrasing this script doesn't recognize), give up and return None rather than guessing.

```
L64-65:
    count = int(match.group(1))
    return count * _UNIT_TO_DAYS[match.group(2)]
```

-> Otherwise convert the matched count to an integer and multiply by that unit's day-fraction from _UNIT_TO_DAYS to get the final approximate age.

```
L66-67: (blank)
```

-> Two blank lines before the next function.

```
L68-70:
def in_age_window(posted_text: str) -> bool:
    age = parse_age_days(posted_text)
    return age is not None and MIN_AGE_DAYS <= age <= MAX_AGE_DAYS
```

-> in_age_window: the single freshness check used everywhere else in the script -- parses the age and reports whether it falls between MIN_AGE_DAYS and MAX_AGE_DAYS inclusive.

```
L71-72: (blank)
```

-> Two blank lines before the Selenium setup function.

```
L73-79:
def build_driver() -> webdriver.Chrome:
    options = ChromeOptions()
    options.add_argument("--headless=new")
    options.add_argument("--no-sandbox")
    options.add_argument("--disable-dev-shm-usage")
    options.add_argument(f"user-agent={USER_AGENT}")
    return webdriver.Chrome(options=options)
```

-> build_driver: configures and returns one headless Chrome instance -- "--headless=new" runs without a visible window, "--no-sandbox" and "--disable-dev-shm-usage" are the standard flags needed for Chrome to run inside a restricted CI container, and the custom user-agent keeps the browser looking like an ordinary desktop visit.

```
L80-81: (blank)
```

-> Two blank lines before the page-fetch function.

```
L82-86:
def fetch_page(driver: webdriver.Chrome, start: int) -> str:
```

```

params = {"keywords": KEYWORDS, "location": LOCATION, "start": start}
url = f"{GUEST_SEARCH_URL}?{urllib.parse.urlencode(params)}"
driver.get(url)
return driver.page_source

```

-> *fetch_page(driver, start): builds the paginated search URL for a given page offset, points the already-open Selenium browser at it with driver.get(), and returns the fully rendered page's HTML -- this is the one function that changed when the project moved from a raw urllib request to a real browser.*

L87-88: (blank)

-> *Two blank lines before the HTML-parsing function.*

L89-114:

```

def parse_cards(html: str):
    soup = BeautifulSoup(html, "html.parser")
    cards = soup.select("div.base-card")
    listings = []
    for card in cards:
        title_el = card.select_one("h3.base-search-card__title")
        company_el = card.select_one("h4.base-search-card__subtitle")
        location_el = card.select_one("span.job-search-card__location")
        time_el = card.select_one("time")
        link_el = card.select_one("a.base-card__full-link")
        if not (title_el and link_el):
            continue
        url = link_el["href"].split("?")[0].strip()
        match = re.search(r"-(\d+)$", url)
        job_id = match.group(1) if match else url
        listings.append(
            {
                "id": job_id,
                "title": title_el.get_text(strip=True),
                "company": company_el.get_text(strip=True) if company_el else "",
                "location": location_el.get_text(strip=True) if location_el else "",
                "posted": time_el.get_text(strip=True) if time_el else "",
                "url": url,
            }
        )
    return listings

```

-> *parse_cards(html): parses one page of results with BeautifulSoup. Selects every "div.base-card" result card, and for each one pulls the title, company, location, relative posted-time, and the canonical job link; skips a card entirely if it's missing a title or a link (defensive against a malformed card); extracts the numeric job id from the trailing digits of the URL (falling back to the whole URL if that pattern doesn't match); and returns the list of listing dicts found on that page.*

L115-116: (blank)

-> Two blank lines before the orchestration function.

```
L117-142:
def fetch_all_listings():
    all_listings = {}
    driver = build_driver()
    try:
        for page in range(MAX_PAGES):
            start = page * PAGE_SIZE
            try:
                html = fetch_page(driver, start)
            except Exception as exc: # noqa: BLE001
                print(f"[warn] fetch failed at start={start}: {exc}", file=sys.stderr)
                break
            listings = parse_cards(html)
            if not listings:
                break
            new_on_this_page = 0
            for job in listings:
                if job["id"] not in all_listings:
                    all_listings[job["id"]] = job
                    new_on_this_page += 1
            if new_on_this_page == 0:
                # LinkedIn started repeating results -- we've reached the end
                break
            time.sleep(REQUEST_DELAY_SECONDS)
    finally:
        driver.quit()
    return list(all_listings.values())
```

-> `fetch_all_listings()`: the main scraping loop. Creates one Chrome driver for the whole run (not one per page), then for each page: fetches it, and stops early if the fetch itself raised an exception (logging a warning to stderr first), or if the page came back with no cards at all. Otherwise merges every listing into an `all_listings` dict keyed by job id (this is the de-duplication step -- a repeated id is simply overwritten with the same data). If an entire page added zero new ids, LinkedIn has started repeating results and the loop stops there rather than wasting further requests. A short sleep between pages keeps requests spaced out. The `finally` block guarantees `driver.quit()` runs even if something above raised, so a failed run never leaves an orphaned Chrome process behind. Returns the deduplicated listings as a plain list.

L143-144: (blank)

-> Two blank lines before the state-loading function.

```
L145-148:
def load_existing():
    if JOBS_JSON.exists():
        return json.loads(JOBS_JSON.read_text(encoding="utf-8"))
```

```
return []
```

-> *load_existing()*: reads the previously written *jobs.json*, or returns an empty list on the very first run when the file doesn't exist yet.

L149-150: (blank)

-> *Two blank lines before the output-writing function.*

L151-166:

```
def write_outputs(jobs):
    jobs_sorted = sorted(jobs, key=lambda j: j["first_seen_utc"], reverse=True)
    JOBS_JSON.write_text(json.dumps(jobs_sorted, indent=2, ensure_ascii=False) + "\n", encoding="utf-8")

    lines = [
        "| Title | Company | Location | Posted | Link | First seen |",
        "| --- | --- | --- | --- | --- | --- |",
    ]
    for j in jobs_sorted:
        lines.append(
            f"| {j['title']} | {j['company']} | {j['location']} | {j['posted']} | "
            f"| [link]({j['url']}) | {j['first_seen_utc']} |"
        )
    JOBS_MD.write_text("\n".join(lines) + "\n", encoding="utf-8")

    build_index_html(jobs_sorted)
```

-> *write_outputs(jobs)*: sorts all tracked jobs newest-first by *first_seen_utc*, writes that exact structure to *jobs.json* as indented JSON, builds the equivalent markdown table and writes *jobs.md*, then calls *build_index_html* to regenerate the live HTML page from the same sorted list.

L167-168: (blank)

-> *Two blank lines before the HTML-building function.*

L169-219:

```
def build_index_html(jobs_sorted):
    generated_at = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC")
    rows = []
    for j in jobs_sorted:
        rows.append(
            "<tr>"
            f"<td>{escape_html(j['title'])}</td>"
            f"<td>{escape_html(j['company'])}</td>"
            f"<td>{escape_html(j['location'])}</td>"
            f"<td>{escape_html(j['posted'])}</td>"
            f"<td><a href=\"{escape_html(j['url'])}\" target=\"_blank\" rel=\"noopener\">Open</a></td>"
            f"<td>{escape_html(j['first_seen_utc'])}</td>"
            "</tr>"
        )
```



```

    )
    html = f"""<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>India DSA Trainer Jobs</title>
<style>
:root {{ color-scheme: light dark; }}
body {{ font-family: -apple-system, Segoe UI, Roboto, sans-serif; margin: 2rem; max-width: 1100px; margin-inline: auto; }}
h1 {{ font-size: 1.4rem; }}
.meta {{ color: #666; margin-bottom: 1.5rem; font-size: 0.9rem; }}
table {{ border-collapse: collapse; width: 100%; font-size: 0.9rem; }}
th, td {{ border: 1px solid #ccc3; padding: 0.5rem 0.6rem; text-align: left; vertical-align: top; }}
th {{ background: #8883; position: sticky; top: 0; }}
tr:hover {{ background: #8881; }}
.wrap {{ overflow-x: auto; }}
</style>
</head>
<body>
<h1>India DSA Trainer Jobs -- LinkedIn (public search, no login)</h1>
<p class="meta">
    {len(jobs_sorted)} listings tracked &middot; last updated {generated_at} &middot;
    keywords="DSA Trainer", location="India" &middot;
    <a href="https://github.com/yogesh889/DSA_Trainer_Online">source repo</a>
</p>
<div class="wrap">
<table>
<thead><tr><th>Title</th><th>Company</th><th>Location</th><th>Posted</th><th>Link</th><th>First seen (UTC)</th></tr></thead>
<tbody>
{' '.join(rows)}
</tbody>
</table>
</div>
</body>
</html>
"""

INDEX_HTML.write_text(html, encoding="utf-8")

```

-> *build_index_html(jobs_sorted)*: builds one HTML table row per job (every field passed through *escape_html* so a title or company containing "&", "<", ">" or a quote can't break the page), then renders a complete static HTML document around those rows -- inline CSS with a ``color-scheme: light dark`` declaration so it adapts to the visitor's OS theme, a sticky table header, a horizontally-scrollable wrapper for narrow screens, a live count and generated-at timestamp in the page's meta line, and a link back to the source repo -- and writes the result to *index.html*.

L220-221: (blank)

-> Two blank lines before the escaping helper.

L222-229:

```
def escape_html(text: str) -> str:
    return (
        (text or "")
        .replace("&", "&amp;")
        .replace("<", "&lt;")
        .replace(">", "&gt;")
        .replace("'", "&quot;")
    )
```

-> `escape_html(text)`: a minimal, dependency-free HTML-escaping helper for the four characters that matter (&, <, >, ") -- used every time job data is interpolated into the generated page.

L230-231: (blank)

-> Two blank lines before the entry-point function.

L232-234:

```
def main():
    existing = load_existing()
    by_id = {j["id"]: j for j in existing}
```

-> `main()` begins: loads whatever was already being tracked from `jobs.json` into `by_id`, a dict keyed by job id, so new results can be merged in below.

L235-236:

```
    fetched = fetch_all_listings()
```

-> Runs the full scrape (`fetch_all_listings`) to get this run's live results, and captures the current time once as an ISO-8601 UTC string to stamp any newly-discovered listings with.

L237:

```
    now = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
```

-> Blank line.

L238-241:

```
    if not fetched:
        print("RESULT: fetch_failed_or_empty new=0 total=%d" % len(existing))
        return
```

-> If the scrape returned nothing at all (for example Chrome/Selenium failed to start, or the network call failed), the run stops here without touching `jobs.json` -- this guards against a transient failure silently wiping out everything that was previously tracked.

L242-243:

```
    new_jobs = []
```

-> Blank line before the merge loop.

L244-263:

```
for job in fetched:
    existing_job = by_id.get(job["id"])
    if existing_job is None:
        # Only start tracking a listing once it's within the window --
        # skip anything under 1 day old (too fresh) or over ~1 month old (stale).
        if not in_age_window(job["posted"]):
            continue
        job["first_seen_utc"] = now
        by_id[job["id"]] = job
        new_jobs.append(job)
    else:
        # Refresh title/company/location/posted from the live page, but
        # keep the original first_seen_utc so history stays accurate.
        existing_job.update(
            title=job["title"],
            company=job["company"],
            location=job["location"],
            posted=job["posted"],
            url=job["url"],
        )
```

-> For every listing just fetched: if its id isn't already tracked, it's only adopted once its posted-text falls inside the freshness window (skipping anything too fresh or already stale), then it's stamped with `first_seen_utc` and added to both `by_id` and the `new_jobs` list that will be reported in this run's log output. If the id was already tracked, its mutable fields (title, company, location, posted text, url) are refreshed in place from the live page, but its original `first_seen_utc` is deliberately left untouched so the tracker's history stays accurate.

L264-268:

```
# Drop anything (new or previously tracked) that's fallen outside the
# 1-day-to-1-month window based on the most recent "posted" text we have.
all_jobs = [j for j in by_id.values() if in_age_window(j["posted"])]
dropped = len(by_id) - len(all_jobs)
```

-> After merging, every tracked listing (old or new) is re-checked against the freshness window using its current posted-text, and anything that has aged out is dropped from the final list -- dropped is just the count of how many, for the log line below.

L269: (blank)

-> Blank line.

L270:

```
write_outputs(all_jobs)
```

-> Persists the final, filtered list via `write_outputs` -- this is the one place `jobs.json` / `jobs.md` / `index.html` actually get written.

L271-272:

```
print(f"RESULT: ok new={len(new_jobs)} total={len(all_jobs)} dropped_out_of_window={dropped}")
```

-> Blank line before the summary output.

L273-274:

```
for j in new_jobs:  
    print(f"NEW: {j['title']} | {j['company']} | {j['location']} | {j['url']}")
```

-> Prints a machine-readable RESULT summary line (counts of new/total/dropped), then one NEW: line per freshly-added listing -- both become part of the GitHub Actions run log.

L275-276: (blank)

-> Two blank lines before the module entry-point guard.

L277-278:

```
if __name__ == "__main__":  
    main()
```

-> Standard `if \_\_name\_\_ == "\_\_main\_\_": main()` guard, so main() only runs when the file is executed directly, not if it were ever imported.

generate_report.py

The email reporter. Runs roughly every 5 days: builds a PDF of new (or all) tracked listings and emails it via Gmail SMTP, using credentials supplied only as GitHub Actions secrets.

```
L1:  
#!/usr/bin/env python3  
-> Shebang.
```

```
L2-12:  
"""  
Builds a PDF summary of India DSA Trainer job listings and emails it to  
the configured recipient. Meant to run every ~5 days  
(see .github/workflows/email-report.yml).  
  
Requires three GitHub Actions secrets (set by the repo owner, never seen by  
Claude or committed to the repo):  
    SMTP_USER      -- the Gmail address to send FROM  
    SMTP_PASS      -- a Gmail App Password for that account (not the login password)  
    RECIPIENT_EMAIL -- the address to send the report TO  
"""
```

-> Module docstring: states this script's purpose (build and email a PDF summary roughly every 5 days) and lists the three GitHub Actions secrets it needs -- explicitly noting these are set by the repo owner and never seen or stored by Claude.

```
L13-20:  
import json  
import os  
import smtplib  
from datetime import datetime, timezone  
from email.mime.application import MIMEApplication  
from email.mime.multipart import MIMEMultipart  
from email.mime.text import MIMEText  
from pathlib import Path
```

-> Imports: json/os/smtplib/datetime/Path for data and email plumbing, the three email.mime classes needed to build a multipart message with a PDF attachment, and fpdf2's FPDF class plus its XPos/YPos enums (used to control the text cursor after each multi_cell call).

```
L21: (blank)
```

-> Blank line before the path/URL constants.

```
L22-28:  
from fpdf import FPDF  
from fpdf.enums import XPos, YPos
```

```
REPO_DIR = Path(__file__).resolve().parent
JOBS_JSON = REPO_DIR / "jobs.json"
STATE_JSON = REPO_DIR / "report_state.json"
PDF_PATH = REPO_DIR / "latest_report.pdf"
```

-> *REPO_DIR* (this file's own directory), the three file paths this script reads or writes (*jobs.json*, *report_state.json*, *latest_report.pdf*), and the two URLs included in every report.

L29-33:

```
LIVE_PAGE_URL = "https://yogesh889.github.io/DSA_Trainer_Online/"
REPO_URL = "https://github.com/yogesh889/DSA_Trainer_Online"
```

-> *Blank lines separating the constants block from the first function.*

L34-37:

```
def load_jobs():
    if JOBS_JSON.exists():
        return json.loads(JOBS_JSON.read_text(encoding="utf-8"))
    return []
```

-> *load_jobs()*: reads *jobs.json*, or returns an empty list if it hasn't been created yet.

L38-39: (blank)

-> *Blank lines.*

L40-43:

```
def load_state():
    if STATE_JSON.exists():
        return json.loads(STATE_JSON.read_text(encoding="utf-8"))
    return {"last_reported_utc": "1970-01-01T00:00:00Z"}
```

-> *load_state()*: reads *report_state.json*, or defaults to the Unix epoch as *last_reported_utc* -- so on the very first run, every currently-tracked job counts as "new".

L44-45: (blank)

-> *Blank lines.*

L46-47:

```
def save_state(state):
    STATE_JSON.write_text(json.dumps(state, indent=2) + "\n", encoding="utf-8")
```

-> *save_state(state)*: writes the (updated) state dict back to *report_state.json*.

L48-49: (blank)

-> *Blank lines.*

L50-52:

```
def pdf_safe(text: str) -> str:
    """Core PDF fonts (Helvetica) only support latin-1; replace anything else."""
    return (text or "").encode("latin-1", "replace").decode("latin-1")
```

-> *pdf_safe(text)*: fpdf2's built-in Helvetica font only supports the latin-1 character set, so any job title/company containing an emoji, a curly quote, or non-Latin script is re-encoded to latin-1 with lossy replacement here, before it ever reaches the PDF -- without this, such a listing would crash the whole report.

L53-54: (blank)

-> *Blank lines.*

```
L55-59:
def build_pdf(all_jobs, new_jobs, generated_at: str) -> Path:
    pdf = FPDF()
    pdf.set_auto_page_break(auto=True, margin=15)
    pdf.add_page()
```

-> *build_pdf()* begins: creates a new FPDF document, turns on automatic page breaks with a 15mm bottom margin (so a long listings section flows onto new pages by itself), and adds the first page.

```
L60-61:
    pdf.set_font("Helvetica", "B", 16)
    pdf.cell(0, 10, "India DSA Trainer Jobs - LinkedIn Report", ln=True)
```

-> *Prints the report's bold 16pt title line.*

```
L62-68:

    pdf.set_font("Helvetica", "", 10)
    pdf.cell(0, 6, f"Generated: {generated_at}", ln=True)
    pdf.cell(0, 6, f"Total listings tracked: {len(all_jobs)}", ln=True)
    pdf.cell(0, 6, f"New since last report: {len(new_jobs)}", ln=True)
    pdf.cell(0, 6, f"Live page: {LIVE_PAGE_URL}", ln=True)
    pdf.cell(0, 6, f"Repo: {REPO_URL}", ln=True)
```

-> *Prints five 10pt meta lines: when the report was generated, how many listings are tracked in total, how many are new since the last report, and the live page + repo URLs.*

```
L69:
    pdf.ln(6)
```

-> *A 6mm vertical gap before the listings section.*

L70: (blank)

-> *Blank line.*

```
L71-72:
    listings = new_jobs if new_jobs else all_jobs
    heading = "New listings since last report" if new_jobs else "No new listings -- showing all currently tracked"
```

-> *Decides what to actually list: the new listings if there are any, otherwise the full tracked set, with a heading string chosen to match.*

L73-75:

```
pdf.set_font("Helvetica", "B", 12)
pdf.cell(0, 8, heading, ln=True)
pdf.ln(2)
```

-> Prints that heading in bold 12pt, followed by a small 2mm gap.

L76: (blank)

-> Blank line.

L77:

```
pdf.set_font("Helvetica", "", 9)
```

-> Switches to plain 9pt body font for the per-listing loop below.

L78-92:

```
for j in listings:
    title = pdf_safe(j.get("title", ""))
    company = pdf_safe(j.get("company", ""))
    location = pdf_safe(j.get("location", ""))
    posted = pdf_safe(j.get("posted", ""))
    url = pdf_safe(j.get("url", ""))

    pdf.set_font("Helvetica", "B", 10)
    pdf.multi_cell(0, 5, title, new_x=XPos.LMARGIN, new_y=YPos.NEXT)
    pdf.set_font("Helvetica", "", 9)
    pdf.multi_cell(0, 5, f"{company} | {location} | posted {posted}", new_x=XPos.LMARGIN, new_y=YPos.NEXT)
    pdf.set_text_color(0, 0, 200)
    pdf.multi_cell(0, 5, url, new_x=XPos.LMARGIN, new_y=YPos.NEXT)
    pdf.set_text_color(0, 0, 0)
    pdf.ln(3)
```

-> The per-listing loop: pulls title/company/location/posted/url through pdf_safe(), prints the title in bold, then a single line combining company, location and posted-time, then the URL rendered in blue (RGB 0,0,200) with the text color reset back to black immediately after so it doesn't bleed into the next entry -- followed by a 3mm gap before the next listing.

L93: (blank)

-> Blank line.

L94-95:

```
pdf.output(str(PDF_PATH))
return PDF_PATH
```

-> Writes the finished document to PDF_PATH (latest_report.pdf) and returns that path to the caller.

L96-97: (blank)

-> Blank lines before the emailing function.

L98-101:


```
def send_email(pdf_path: Path, new_count: int, total_count: int):
    smtp_user = os.environ["SMTP_USER"]
    smtp_pass = os.environ["SMTP_PASS"]
    recipient_email = os.environ["RECIPIENT_EMAIL"]
```

-> *send_email()* begins: reads the three required secrets out of the environment -- this is the only place credentials touch the script, and they only ever exist as process environment variables injected by the GitHub Actions workflow, never written to disk or logged.

L102-106:

```
msg = MIMEMultipart()
msg["From"] = smtp_user
msg["To"] = recipient_email
msg["Subject"] = f"India DSA Trainer jobs: {new_count} new ({total_count} total tracked)"
```

-> Builds a MIMEMultipart message with From/To headers and a Subject line that already summarizes the new/total counts, so they're visible in an inbox without opening the email.

L107-114:

```
body = (
    f"{new_count} new India DSA Trainer listing(s) found since the last report.\n"
    f"{total_count} total listings currently tracked.\n\n"
    f"Live page (always current): {LIVE_PAGE_URL}\n"
    f"Source repo: {REPO_URL}\n\n"
    "Full details attached as PDF."
)
```

-> Composes the plain-text email body: the new/total counts again, both URLs, and a note that the full detail is in the attached PDF.

L115:

```
msg.attach(MIMEText(body, "plain"))
```

-> Attaches that text body to the message.

L116: (blank)

-> Blank line.

L117-120:

```
with open(pdf_path, "rb") as f:
    part = MIMEApplication(f.read(), _subtype="pdf")
    part.add_header("Content-Disposition", "attachment", filename=pdf_path.name)
    msg.attach(part)
```

-> Re-opens the PDF just written, wraps its bytes as a MIMEApplication attachment tagged with the right filename, and attaches it to the message.

L121: (blank)

-> Blank line.

L122-124:

```
with smtplib.SMTP_SSL("smtp.gmail.com", 465) as server:
    server.login(smtp_user, smtp_pass)
    server.sendmail(smtp_user, [recipient_email], msg.as_string())
```

-> Opens an SSL connection straight to Gmail's SMTP server on port 465, logs in with the app password, and sends the finished message to the recipient.

L125-126: (blank)

-> Blank lines before main().

L127-130:

```
def main():
    all_jobs = load_jobs()
    state = load_state()
    last_reported = state.get("last_reported_utc", "1970-01-01T00:00:00Z")
```

-> main() begins: loads every currently-tracked job and the saved report state, pulling out last_reported_utc.

L131: (blank)

-> Blank line.

L132:

```
new_jobs = [j for j in all_jobs if j.get("first_seen_utc", "") > last_reported]
```

-> Computes new_jobs as every job whose first_seen_utc string sorts after last_reported_utc -- this string comparison works correctly because both are ISO-8601 UTC timestamps, which sort lexicographically in the same order they occur chronologically.

L133:

```
generated_at = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M UTC")
```

-> Captures the current time as the report's generated-at display string.

L134: (blank)

-> Blank line.

L135-136:

```
pdf_path = build_pdf(all_jobs, new_jobs, generated_at)
send_email(pdf_path, len(new_jobs), len(all_jobs))
```

-> Builds the PDF, then immediately emails it along with the new/total counts.

L137: (blank)

-> Blank line.

L138-139:

```
state["last_reported_utc"] = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
save_state(state)
```

-> Updates last_reported_utc in the state dict to now, and saves it -- the workflow commits this change afterward, so the next run's new_jobs comparison only picks up what's genuinely new since this email went out.

L140: (blank)

-> *Blank line.*

L141:

```
print(f"RESULT: emailed new={len(new_jobs)} total={len(all_jobs)}")
```

-> *Prints a machine-readable RESULT summary line for the workflow log.*

L142-143: (blank)

-> *Blank lines before the entry-point guard.*

L144-145:

```
if __name__ == "__main__":  
    main()
```

-> *Standard `if \_\_name\_\_ == "\_\_main\_\_": main()` guard.*

.github/workflows/dsa-trainer-job-watch.yml

The GitHub Actions workflow that schedules and runs scrape.py, and commits its output back to the repository when it changes.

L1:
name: india-dsa-trainer-job-watch

-> The workflow's display name, as it appears in the repo's Actions tab.

L2: (blank)

-> Blank line.

L3-6:
on:
 schedule:
 - cron: "13 */5 * * *"
 workflow_dispatch: {}

-> Triggers: a cron schedule ("13 */5 * * *" -- at minute 13 of every 5th hour, so roughly every 5 hours around the clock) and workflow_dispatch, which adds the manual "Run workflow" button in the Actions tab.

L7: (blank)

-> Blank line.

L8-9:
permissions:
 contents: write

-> Grants the workflow's default GITHUB_TOKEN write access to repository contents -- required because the last step below commits and pushes.

L10: (blank)

-> Blank line.

L11-13:
jobs:
 scrape:
 runs-on: ubuntu-latest

-> Defines a single job named "scrape", running on GitHub's standard ubuntu-latest runner image.

L14:
 steps:

-> Start of the step list.

L15:
 - uses: actions/checkout@v4

-> Checks out the repository so later steps can read and write its files.

L16: (blank)

-> Blank line.

L17-19:

```
- uses: actions/setup-python@v5
  with:
    python-version: "3.12"
```

-> Installs Python 3.12 on the runner via the official setup-python action.

L20: (blank)

-> Blank line.

L21-22:

```
- name: Install dependencies
  run: pip install beautifulsoup4 selenium
```

-> Installs the scraper's two runtime dependencies directly (beautifulsoup4 for HTML parsing, selenium for browser automation); Chrome itself and a matching driver are supplied by the runner image together with Selenium's own driver-manager, so no separate browser-install step is needed.

L23: (blank)

-> Blank line.

L24-25:

```
- name: Run scraper
  run: python scrape.py
```

-> Runs scrape.py -- this is the step that does the actual fetch/parse/dedupe/write work.

L26: (blank)

-> Blank line.

L27-37:

```
- name: Commit and push if changed
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    if ! git diff --quiet -- jobs.json jobs.md index.html; then
      git add jobs.json jobs.md index.html
      git commit -m "Update India DSA Trainer job listings"
      git push
    else
      echo "No changes to commit."
    fi
```

-> Commit-and-push step: configures a bot git identity, then only stages, commits, and pushes jobs.json/jobs.md/index.html if `git diff --quiet` reports they actually changed -- avoiding an empty commit on runs where nothing new was found -- and otherwise just logs that nothing changed.

.github/workflows/email-report.yml

The GitHub Actions workflow that schedules and runs generate_report.py, supplying the SMTP/recipient secrets and committing the updated report state.

L1:
name: dsa-trainer-email-report

-> The workflow's display name -- distinct from the job-watch workflow so both show up clearly in the Actions tab.

L2: (blank)

-> Blank line.

L3-6:
on:
 schedule:
 - cron: "0 3 */5 * *"
 workflow_dispatch: {}

-> Triggers: a cron schedule ("0 3 */5 * *" -- 03:00 UTC on days 1, 6, 11, 16, 21, 26 and 31 of each month) that approximates "every 5 days" -- standard cron can't express a true rolling interval across month boundaries, so this occasionally fires a day or two early at a month end -- plus workflow_dispatch for a manual run.

L7: (blank)

-> Blank line.

L8-9:
permissions:
 contents: write

-> Grants write access to repository contents, needed to commit the updated report_state.json.

L10: (blank)

-> Blank line.

L11-13:
jobs:
 report:
 runs-on: ubuntu-latest

-> Defines a single job named "report", on ubuntu-latest.

L14:
 steps:

-> Start of the step list.

L15:
 - uses: actions/checkout@v4

-> Checks out the repository.

L16: (blank)

-> Blank line.

L17-19:

```
- uses: actions/setup-python@v5
  with:
    python-version: "3.12"
```

-> Installs Python 3.12.

L20: (blank)

-> Blank line.

L21-22:

```
- name: Install dependencies
  run: pip install -r requirements.txt
```

-> Installs every dependency listed in requirements.txt -- this workflow only actually needs fpdf2, but reusing one shared requirements file keeps the two workflows simple and consistent.

L23: (blank)

-> Blank line.

L24-29:

```
- name: Generate PDF and send email
  env:
    SMTP_USER: ${ secrets.SMTP_USER }
    SMTP_PASS: ${ secrets.SMTP_PASS }
    RECIPIENT_EMAIL: ${ secrets.RECIPIENT_EMAIL }
  run: python generate_report.py
```

-> Runs generate_report.py with the three secrets (SMTP_USER, SMTP_PASS, RECIPIENT_EMAIL) injected as environment variables for just this step -- this is the only place those credentials exist anywhere in the pipeline.

L30: (blank)

-> Blank line.

L31-41:

```
- name: Commit updated report state
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "github-actions[bot]@users.noreply.github.com"
    git add report_state.json
    if ! git diff --cached --quiet -- report_state.json; then
      git commit -m "Update report state after emailing summary"
      git push
    else
      echo "No state change to commit."
```

fi

-> Commit step: configures a bot git identity, stages `report_state.json`, and only commits+pushes if it actually changed (mirrors the same guarded-commit pattern used in the job-watch workflow), so the timestamp used for next run's "new since last report" comparison is always the one that was really emailed.

requirements.txt

The three third-party packages both scripts depend on.

L1:

`beautifulsoup4`

-> *beautifulsoup4* -- HTML parsing library; *scrape.py*'s *parse_cards()* uses it to pull structured fields out of each result card.

L2:

`fpdf2`

-> *fpdf2* -- PDF-generation library; used by *generate_report.py* to build the emailed summary, and by this document's own generator.

L3:

`selenium`

-> *selenium* -- browser-automation library; *scrape.py* uses it to drive a real headless Chrome browser against LinkedIn's public guest search, instead of a raw HTTP request.

Generated files (not walked through line-by-line)

jobs.json

Generated data -- the structured record of every currently-tracked listing. Rewritten by scrape.py's write_outputs() on every run; not hand-authored.

jobs.md

Generated data -- the same listings as a human-readable markdown table.

index.html

Generated output -- the static live page served by GitHub Pages, rebuilt from jobs_sorted by build_index_html() on every scrape.py run.

report_state.json

Generated state -- just {"last_reported_utc": <timestamp>}, rewritten by generate_report.py after every emailed report.

latest_report.pdf

Generated output -- the most recent emailed PDF, rebuilt by generate_report.py each time it runs.